

LAPORAN PRAKTIKUM PEKAN 5

STRUKTUR DATA

Disusun Oleh:

Syasya Halwa Gazwani

(2511531018)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Alvero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh,

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW. yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini. Penulis mengucapkan terima kasih kepada bapak Dr. Wahyudi, S.T, M.T. selaku dosen mata kuliah Struktur Data yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan mata kuliah yang penulis tekuni. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga penulis dapat menyelesaikan Laporan Praktikum ini.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum	2
BAB II PEMBAHASAN	3
DASAR TEORI	3
2.1 NodeSLL_2511531018	4
2.1.1 Kode Program	4
2.1.2 Langkah Kerja	4
2.2 TambahSLL_2511531018	4
2.2.1. Kode Program	4
2.2.2. Langkah Kerja	7
2.2.3 Output	10
2.3 PencarianSLL_2511531018	10
2.3.1 Kode Program	10
2.3.2 Langkah Kerja	11
2.4.3 Output	12
2.4 HapusSLL_2511531018	13
2.4.1 Kode Program	13
2.4.2 Langkah Kerja	15
2.4.3 Output	18
2.6 Tugas.....	19

2.6.1	Class Pasien_2511531018.....	19
2.6.2	Class RumahSakit_2511531018	19
2.6.3	Output	24
BAB III PENUTUP		28
DAFTAR PUSTAKA.....		29

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, struktur data digunakan untuk mengatur dan mengelola data agar dapat diakses serta dimanipulasi dengan efisien. Salah satu struktur data yang penting adalah Single Linked List (SLL), yaitu kumpulan node yang saling terhubung melalui pointer. Setiap node terdiri dari dua bagian utama, yaitu data dan referensi (next) yang menunjuk ke node berikutnya. Pembentukan dasar dari struktur ini direpresentasikan melalui class NodeSLL, yang berfungsi sebagai elemen penyusun linked list. Node ini menjadi fondasi utama dalam membangun hubungan antar data secara berantai.

Untuk melakukan manipulasi data, digunakan class TambahSLL yang berfungsi untuk menambahkan node ke dalam linked list, baik di bagian depan, belakang, maupun pada posisi tertentu. Proses ini menunjukkan bagaimana data dapat dimasukkan secara dinamis tanpa perlu menggeser elemen lain seperti pada array. Selain itu, terdapat class PencarianSLL yang digunakan untuk melakukan pencarian data dalam linked list. Proses pencarian dilakukan dengan menelusuri setiap node secara berurutan hingga data yang dicari ditemukan atau hingga mencapai akhir list. Selanjutnya, class HapusSLL digunakan untuk menghapus node dari linked list, baik pada bagian awal, akhir, maupun pada posisi tertentu. Proses ini melibatkan perubahan pointer antar node sehingga struktur list tetap terhubung dengan benar setelah penghapusan dilakukan.

Dengan memahami konsep NodeSLL serta operasi penambahan, pencarian, dan penghapusan pada Single Linked List, dapat diperoleh pemahaman yang lebih mendalam mengenai pengelolaan data secara dinamis dalam pemrograman.

1.2 Tujuan

1.2.1 Memahami konsep dasar struktur data Single Linked List.

- 1.2.2 Mampu mengimplementasikan operasi penambahan node pada SLL (di depan, belakang, dan posisi tertentu).
- 1.2.3 Mampu mengimplementasikan operasi penghapusan node pada SLL (di awal, akhir, dan posisi tertentu).
- 1.2.4 Mampu melakukan pencarian data dalam Single Linked List.
- 1.2.5 Memahami cara kerja pointer dalam menghubungkan antar node pada linked list.

1.3 Manfaat Praktikum

- 1.3.1 Mahasiswa dapat memahami cara kerja struktur data dinamis secara lebih mendalam.
- 1.3.2 Mahasiswa mampu mengelola data secara fleksibel tanpa bergantung pada ukuran tetap seperti pada array.
- 1.3.3 Melatih kemampuan logika dalam mengatur hubungan antar node menggunakan pointer.
- 1.3.4 Menjadi dasar untuk mempelajari struktur data yang lebih kompleks seperti doubly linked list, stack, queue, dan tree.
- 1.3.5 Membantu dalam pengembangan program yang membutuhkan manipulasi data secara efisien

BAB II PEMBAHASAN

DASAR TEORI

Struktur data merupakan cara untuk menyimpan, mengatur, dan mengelola data di dalam komputer agar data dapat diproses secara efisien. Penggunaan struktur data yang tepat dapat mempermudah proses pengolahan data seperti penambahan, penghapusan, pencarian, dan penelusuran data. Salah satu struktur data linear yang sering digunakan adalah Single Linked List. Single Linked List adalah struktur data yang terdiri dari kumpulan node yang saling terhubung satu arah. Setiap node memiliki dua bagian utama, yaitu data dan pointer next yang berfungsi untuk menunjuk node berikutnya. Node pertama disebut head, sedangkan node terakhir akan menunjuk ke null yang menandakan akhir dari linked list.

Single Linked List memiliki sifat dinamis karena ukuran data dapat bertambah atau berkurang selama program berjalan. Hal ini berbeda dengan array yang ukuran datanya bersifat tetap. Dengan menggunakan linked list, proses penambahan dan penghapusan data dapat dilakukan dengan lebih fleksibel karena tidak memerlukan pergeseran elemen seperti pada array. Beberapa operasi dasar pada Single Linked List antara lain insert, delete, traversal, dan searching. Operasi insert digunakan untuk menambahkan node baru ke dalam linked list, baik di awal, akhir, maupun posisi tertentu. Operasi delete digunakan untuk menghapus node dari linked list. Operasi traversal dilakukan dengan menelusuri seluruh node dari head hingga node terakhir untuk menampilkan data. Sedangkan searching digunakan untuk mencari data tertentu dengan menelusuri node satu per satu sampai data ditemukan atau mencapai akhir linked list.

Dalam implementasinya, Single Linked List menggunakan pointer untuk menghubungkan setiap node. Pointer next menyimpan alamat node berikutnya sehingga node-node pada linked list dapat saling terhubung dan membentuk suatu rangkaian data yang berurutan. Implementasi struktur data dapat dibuat menggunakan Java. Java merupakan bahasa pemrograman berorientasi objek yang mendukung penggunaan class dan object sehingga mempermudah pembuatan node serta pengelolaan operasi pada Single Linked List.

2.1 NodeSLL_2511531018

2.1.1 Kode Program

```
package pekan5_2511531018;

public class NodeSLL_2511531018 {
    // node bagian data
    int data_1018;
    // Pointed ke node berikutnya
    NodeSLL_2511531018 next_1018;
    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511531018(int data_1018)
    {
        this.data_1018=data_1018;
        this.next_1018=null;
    }
}
```

Gambar 2.1.1 Kode program NodeSLL_2511531018.

2.1.2 Langkah Kerja

2.1.2.1 Data_1018 untuk menyimpan isi data seperti angka dan next_1018 sebagai petunjuk ke node berikutnya.

```
// node bagian data
int data_1018;
// Pointed ke node berikutnya
NodeSLL_2511531018 next_1018;
```

Gambar 2.1.2.1 Variabel data dan pointer.

2.1.2.2 Fungsi yang otomatis jalan saat bikin node baru.

```
// Konstruktor menginisialisasi node dengan data
public NodeSLL_2511531018(int data_1018)
{
```

Gambar 2.1.2.2 Konstruktor saat node dibuat.

2.1.2.3 Data yang dimasukkan disimpan ke node.

```
this.data_1018=data_1018;
this.next_1018=null;
```

Gambar 2.1.2.3 Isi konstruktor.

2.2 TambahSLL_2511531018

2.2.1. Kode Program

```
package pekan5_2511531018;

public class TambahSLL_2511531018 {
```



```

    public static NodeSLL_2511531018
insertAtFront_1018(NodeSLL_2511531018 head_1018, int
value_1018) {
    NodeSLL_2511531018 new_node_1018 = new
NodeSLL_2511531018(value_1018);
    new_node_1018.next_1018=head_1018;
    return new_node_1018;
}
// fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511531018
insertAtEnd_1018(NodeSLL_2511531018 head_1018, int
value_1018) {
    // buat sebuah node dengan sebuah nilai
    NodeSLL_2511531018 newNode_1018=new
NodeSLL_2511531018(value_1018);
    // jika list kosong maka node jadi head
    if (head_1018 == null) {
        return newNode_1018;
    }
    //simpan ead ke variabel sementara
    NodeSLL_2511531018 last_1018 = head_1018;
    // telusuri ke node akhir
    while (last_1018.next_1018 != null) {
        last_1018=last_1018.next_1018;
    }
    //ubah pointer
    last_1018.next_1018 = newNode_1018;
    return head_1018;
}
    static NodeSLL_2511531018 GetNode_1018(int data_1018) {
        return new NodeSLL_2511531018(data_1018);
    }
    static NodeSLL_2511531018 insertPos_1018(NodeSLL_2511531018
headNode, int position_1018, int value_1018) {
        NodeSLL_2511531018 head_1018 = headNode;
        if(position_1018 < 1)
            System.out.print("Invalid position");
        if(position_1018 == 1) {
            NodeSLL_2511531018 new_node_1018 = new
NodeSLL_2511531018(value_1018);
            new_node_1018.next_1018=head_1018;
            return new_node_1018;
        } else {
            while (position_1018-- != 0) {
                if (position_1018==1) {
                    NodeSLL_2511531018 newNode_1018
= GetNode_1018(value_1018);

                    newNode_1018.next_1018=headNode.next_1018;
                    headNode.next_1018 =
newNode_1018;

                    break;
                }
                headNode=headNode.next_1018;
            }
            if (position_1018 != 1)
                System.out.print("Posisi di luar jangkauan");

```

```

        return head_1018;
    }
}

public static void printList_1018(NodeSLL_2511531018
head_1018) {
    NodeSLL_2511531018 curr_1018 = head_1018;
    while (curr_1018.next_1018 != null) {
        System.out.print(curr_1018.data_1018+"-->");
        curr_1018 = curr_1018.next_1018;
    }
    if (curr_1018.next_1018==null) {
        System.out.print(curr_1018.data_1018);
        System.out.println();
    }
}

public static void main(String[] args) {
    //buat linked list 2->3->5->6
    NodeSLL_2511531018 head_1018 = new
NodeSLL_2511531018(2);
    head_1018.next_1018 = new
NodeSLL_2511531018(3);
    head_1018.next_1018.next_1018 = new
NodeSLL_2511531018(5);
    head_1018.next_1018.next_1018.next_1018 = new
NodeSLL_2511531018(6);
    // cetak list asli
    System.out.print("Senarai berantai awal : ");
    printList_1018(head_1018);
    //tambahkan node baru didepan
    System.out.print("tambah 1 simpul di depan:
");

    int data_1018 =1;
    head_1018 = insertAtFront_1018(head_1018,
data_1018);

    //cetak update list
    printList_1018(head_1018);
    //tambahkan node baru di belakang
    System.out.print("tambah 1 simpul di
belakang: ");
    int data2_1018 = 7;
    head_1018 = insertAtEnd_1018(head_1018,
data2_1018);

    //cetak update list
    printList_1018(head_1018);
    System.out.print("tambah 1 simpul ke data 4:
");

    int data3_1018 = 4;
    int pos_1018 = 4;
    head_1018 = insertPos_1018(head_1018,
pos_1018, data3_1018);
    //cetak update list
    printList_1018(head_1018);
}
}

```

Gambar 2.2.1 Kode program TambahSLL_2511531018.

2.2.2. Langkah Kerja

- 2.2.2.1 Fungsi ini untuk menambahkan node di depan list dengan membuat node baru dengan isi value_1018, lalu node baru diarahkan ke head lama dan node baru menjadi head baru

```
public static NodeSLL_2511531018
insertAtFront_1018(NodeSLL_2511531018 head_1018, int
value_1018) {
    NodeSLL_2511531018 new_node_1018 = new
NodeSLL_2511531018(value_1018);
    new_node_1018.next_1018=head_1018;
    return new_node_1018;
}
```

Gambar 2.2.2.1 Fungsi insertAtFront_1018.

- 2.2.2.2 Fungsi untuk menambahkan node di akhir list dengan membuat node baru.

```
public static NodeSLL_2511531018
insertAtEnd_1018(NodeSLL_2511531018 head_1018, int
value_1018) {
    // buat sebuah node dengan sebuah nilai
    NodeSLL_2511531018 newNode_1018=new
NodeSLL_2511531018(value_1018);
    // jika list kosong maka node jadi head
    if (head_1018 == null) {
        return newNode_1018;
    }
```

Gambar 2.2.2.2 Fungsi insertAtEnd_1018.

- 2.2.2.3 Jika list kosong, node langsung menjadi head dan variabel membantu membuat jalan dari head. Loop sampai ke node terakhir, node terakhir diarahkan ke node baru, dan head tetap (tidak berubah).

```
//simpan ead ke variabel sementara
NodeSLL_2511531018 last_1018 = head_1018;
// telusuri ke node akhir
while (last_1018.next_1018 != null) {
    last_1018=last_1018.next_1018;
}
//ubah pointer
last_1018.next_1018 = newNode_1018;
return head_1018;
}
```

Gambar 2.2.2.3 Fungsi insertAtEnd_1018.

- 2.2.2.4 Fungsi sederhana untuk membuat node baru agar lebih ringkas dipanggil pada fungsi lain.

```
static NodeSLL_2511531018 GetNode_1018(int data_1018)
{
    return new NodeSLL_2511531018(data_1018);
}
```

Gambar 2.2.2.4 Fungsi GetNode_1018.

- 2.2.2.5 Fungsi untuk menambahkan node di posisi tertentu dengan menyimpan head awal, jika posisi kurang dari 1 berarti tidak valid, jika posisi 1 berarti sama seperti insert depan..

```
static NodeSLL_2511531018
insertPos_1018(NodeSLL_2511531018 headNode, int
position_1018, int value_1018) {
    NodeSLL_2511531018 head_1018 = headNode;
    if(position_1018 < 1)
        System.out.print("Invalid position");
    if(position_1018 == 1) {
        NodeSLL_2511531018 new_node_1018 = new
NodeSLL_2511531018(value_1018);
        new_node_1018.next_1018=head_1018;
        return new_node_1018;
    }
}
```

Gambar 2.2.2.5 Fungsi insertPos_1018.

- 2.2.2.6 Jika posisi bukan 1, loop mengurangi posisi, saat sudah sampai sebelum posisi target, membuat node baru, dan node baru diarahkan ke node setelahnya, node sebelumnya diarahkan ke node baru, dan keluar loop.

```
while (position_1018-- != 0) {
    if (position_1018==1) {
        NodeSLL_2511531018
newNode_1018 = GetNode_1018(value_1018);

        newNode_1018.next_1018=headNode.next_1018;
        headNode.next_1018 =
newNode_1018;
        break;
    }
    headNode=headNode.next_1018;
}
```

Gambar 2.2.2.6 Fungsi insertPos_1018.

- 2.2.2.7 Jika posisi melebihi panjang list berarti error, balikin ke head awal.

```
if (position_1018 != 1)
    System.out.print("Posisi di luar jangkauan");
return head_1018;
}
```

Gambar 2.2.2.7 Fungsi insertPos_1018.

- 2.2.2.8 Fungsi untuk menampilkan isi linked list, mulai dari head, loop sampai sebelum node terakhir, print data dan panah, geser ke node berikutnya, jika sudah node terakhir, print tanpa panah..

```
public static void printList_1018(NodeSLL_2511531018
head_1018) {
    NodeSLL_2511531018 curr_1018 = head_1018;
    while (curr_1018.next_1018 != null) {
        System.out.print(curr_1018.data_1018+"-
->");
        curr_1018 = curr_1018.next_1018;
    }
    if (curr_1018.next_1018==null) {
        System.out.print(curr_1018.data_1018);
        System.out.println();
    }
}
```

Gambar 2.2.2.8 Fungsi printList_1018.

- 2.2.2.9 Membuat linked list awal. Node pertama: 2, sehingga list: 2 => 3 => 5 => 6.

```
NodeSLL_2511531018 head_1018 = new
NodeSLL_2511531018(2);
head_1018.next_1018 = new
NodeSLL_2511531018(3);
head_1018.next_1018.next_1018 = new
NodeSLL_2511531018(5);
head_1018.next_1018.next_1018.next_1018
= new NodeSLL_2511531018(6);
```

gambar 2.2.2.9 Fungsi main.

- 2.2.2.10 Cetak list awal dan menambahkan di depan sehingga menjadi 1 => 2 => 3 => 5 => 6.

```
System.out.print("Senarai berantai awal : ");
printList_1018(head_1018);
//tambahkan node baru didepan
System.out.print("tambah 1 simpul di
depan: ");
int data_1018 =1;
head_1018 =
insertAtFront_1018(head_1018, data_1018);
```

Gambar 2.2.2.10 Fungsi main.

- 2.2.2.11 Menambahkan di belakang sehingga menjadi Menambahkan di depan sehingga menjadi 1 => 2 => 3 => 5 => 6 => 7.

```
System.out.print("tambah 1 simpul di belakang: ");
int data2_1018 = 7;
head_1018 = insertAtEnd_1018(head_1018,
data2_1018);
```

Gambar 2.2.2.11 Fungsi main.

2.2.2.12 Menambahkan angka 4 pada posisi ke-4.

```
System.out.print("tambah 1 simpul ke data 4: ");
    int data3_1018 = 4;
    int pos_1018 = 4;
    head_1018 = insertPos_1018(head_1018,
pos_1018, data3_1018);
```

Gambar 2.2.2.12 Fungsi main.

2.2.3 Output

```
Senarai berantai awal : 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

Gambar 2.2.3 Output TambahSLL_2511531018.

2.3 PencarianSLL_2511531018

2.3.1 Kode Program

```
package pekan5_2511531018;

public class PencarianSSL_2511531018 {
    static boolean searchKey_1018(NodeSLL_2511531018 head_1018,
int key_1018) {
        NodeSLL_2511531018 curr_1018 = head_1018;
        while (curr_1018 != null) {
            if (curr_1018.data_1018 == key_1018)
                return true;
            curr_1018 = curr_1018.next_1018; }
        return false; }
    public static void traversal(NodeSLL_2511531018 head_1018)
{
    //mulai dari head
    NodeSLL_2511531018 curr_1018 = head_1018;
    //telusuri sampai pointer null
    while (curr_1018 != null) {
        System.out.print(" " + curr_1018.data_1018);
        curr_1018 = curr_1018.next_1018; }
    System.out.println(); }
    public static void main(String[] args) {
        NodeSLL_2511531018 head_1018 = new
NodeSLL_2511531018(14);
        head_1018.next_1018 = new NodeSLL_2511531018(21);
        head_1018.next_1018.next_1018 = new
NodeSLL_2511531018(13);
        head_1018.next_1018.next_1018.next_1018 = new
NodeSLL_2511531018(30);
        head_1018.next_1018.next_1018.next_1018.next_1018 =
new NodeSLL_2511531018(10);
        System.out.print("Penelusuran SLL : ");
```

```

    traversal(head_1018);
    //data yang akan dicari
    int key_1018 = 30;
    System.out.print("cari data " +key_1018+ " = ");
    if (searchKey_1018(head_1018, key_1018))
        System.out.println("Ketemu");
    else
        System.out.println("tidak ada");
}
}

```

Gambar 2.3.1 Kode program PencarianSLL_2511531018.

2.3.2 Langkah Kerja

2.3.2.1 Fungsi dibuat untuk mencari apakah suatu nilai ada di linked list atau tidak. Alurnya mulai dari head (node pertama), selama node belum habis (belum null), terus dicek satu-satu. Jika data di node sama dengan yang dicari maka langsung ketemu (true). Jika belum ketemu maka pindah ke node berikutnya. Jika sudah sampai akhir tapi tidak ada, maka false (tidak ditemukan).

```

static boolean searchKey_1018(NodeSLL_2511531018
head_1018, int key_1018) {
    NodeSLL_2511531018 curr_1018 = head_1018;
    while (curr_1018 != null) {
        if (curr_1018.data_1018 == key_1018)
            return true;
        curr_1018 = curr_1018.next_1018; }
    return false; }

```

Gambar 2.3.2.1 Fungsi searchKey_1018.

2.3.2.2 Fungsi ini dibuat untuk menampilkan semua isi linked list. Alurnya mulai dari head, selama node masih ada, tampilkan data. Geser ke node berikutnya lalu pindah aris setelah selesai.

```

public static void traversal(NodeSLL_2511531018
head_1018) {
    //mulai dari head
    NodeSLL_2511531018 curr_1018 = head_1018;
    //telusuri sampai pointer null
    while (curr_1018 != null) {
        System.out.print(" " +
curr_1018.data_1018);
        curr_1018 = curr_1018.next_1018; }
    System.out.println(); }

```

Gambar 2.3.2.2 Fungsi traversal (menampilkan isi list).

2.3.2.3 Membuat linked list, node pertama: 14, sehingga linked list:
14 => 21 => 13 => 30 => 10.

```
NodeSLL_2511531018 head_1018 = new
NodeSLL_2511531018(14);
    head_1018.next_1018 = new
NodeSLL_2511531018(21);
    head_1018.next_1018.next_1018 = new
NodeSLL_2511531018(13);
    head_1018.next_1018.next_1018.next_1018 = new
NodeSLL_2511531018(30);

    head_1018.next_1018.next_1018.next_1018.next_1018 =
new NodeSLL_2511531018(10);
```

Gambar 2.3.2.3 Bagian main (jalannya program).

2.3.2.4 Menampilkan isi list.

```
System.out.print("Penelusuran SLL : ");
    traversal(head_1018);
```

Gambar 2.3.2.4 Bagian main (jalannya program).

2.3.2.5 Menentukan data yang ingin dicari dan menampilkan teks dilayar. Program mulai memanggil fungsi pencarian mulai dari node pertama (head) dan dicek satu persatu, jika ketemu, fungsi langsung mengembalikan true. Jika data tidak ditemukan, maka false.

```
int key_1018 = 30;
    System.out.print("cari data " +key_1018+ " =
");
    if (searchKey_1018(head_1018, key_1018))
        System.out.println("Ketemu");
    else
        System.out.println("tidak ada");
}
```

Gambar 2.3.2.5

2.4.3 Output

```
Penelusuran SLL : 14 21 13 30 10
cari data 30 = Ketemu
```

Gambar 2.3.3 Output PencarianSLL_2511531018.

2.4 HapusSLL_2511531018

2.4.1 Kode Program

```
package pekan5_2511531018;

public class HapusSLL_2511531018 {
    //fungsi untuk menghapus head
    public static NodeSLL_2511531018
deleteHead_1018(NodeSLL_2511531018 head_1018) {
    //jika SLL kosong
    if (head_1018 == null)
        return null;
    //pindahkan head ke node berikutnya
    head_1018 = head_1018.next_1018;
    //return head baru
    return head_1018; }
    //fungsi menghapus node terakhir SLL
    public static NodeSLL_2511531018
removeLastNode_1018(NodeSLL_2511531018 head_1018) {
    //jika list kosong, return null
    if (head_1018 == null) {
        return null;
    }
    //jika list satu node, hapus node dan return
    null

    if (head_1018.next_1018 == null) {
        return null;
    }
    //temukan node terakhir ke dua
    NodeSLL_2511531018 secondLast_1018 =
head_1018;
    while (secondLast_1018.next_1018.next_1018 !=
null) {
        secondLast_1018 =
secondLast_1018.next_1018;
    }
    //hapus node terakhir
    secondLast_1018.next_1018 = null;
    return head_1018; }

    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511531018
deleteNode_1018(NodeSLL_2511531018 head_1018, int
position_1018) {
    NodeSLL_2511531018 temp_1018 =
head_1018;
    NodeSLL_2511531018 prev_1018 = null;
    //jika linked list null
    if (temp_1018 == null)
        return head_1018;
    //kasus 1: head dihapus
    if (position_1018 == 1) {
        head_1018 = temp_1018.next_1018;
        return head_1018; }
    //kasus 2: menghapus node
    ditengah
```

```

//telusuri ke node yang dihapus
for (int i_1018 = 1; temp_1018
!= null && i_1018 < position_1018; i_1018++) {
    prev_1018 = temp_1018;
    temp_1018 =
temp_1018.next_1018; }

//jika ditemukan, hapus node
if (temp_1018 != null) {
    prev_1018.next_1018 =
temp_1018.next_1018;
} else {
    System.out.println("Data
tidak ada"); }

return head_1018; }

// fungsi mencetak SLL
public static void
printList_1018(NodeSLL_2511531018 head_1018) {
    NodeSLL_2511531018
curr_1018 = head_1018;
while (curr_1018.next_1018
!= null) {

    System.out.print(curr_1018.data_1018+"-->");
    curr_1018 =
curr_1018.next_1018; }

    if (curr_1018.next_1018 ==
null) {

        System.out.print(curr_1018.data_1018); }

        System.out.println(); }

//kelas main
public static void main(String[]
args) {

    //buat SLL 1 -> 2 -> 3 ->
4 -> 5 -> 6 -> null

    NodeSLL_2511531018
head_1018 = new NodeSLL_2511531018(1);
    head_1018.next_1018 = new
NodeSLL_2511531018(2);

    head_1018.next_1018.next_1018 = new NodeSLL_2511531018(3);

    head_1018.next_1018.next_1018.next_1018 = new
NodeSLL_2511531018(4);

    head_1018.next_1018.next_1018.next_1018.next_1018 = new
NodeSLL_2511531018(5);

    head_1018.next_1018.next_1018.next_1018.next_1018.next_1018
= new NodeSLL_2511531018(6);

    // cetak list awal
    System.out.println("list
awal: ");

    printList_1018(head_1018);
    //hapus head

```

```

deleteHead_1018(head_1018);
    head_1018 =
    System.out.println("List
    setelah simpul terakhir di hapus: ");
    printList_1018(head_1018);
    //hapus node terakhir
    head_1018 =
removeLastNode_1018(head_1018);
    System.out.println("List
    setelahsimpul terakhir di hapus: ");
    printList_1018(head_1018);
    //Deleting node at
position 2
    int position_1018 = 2;
    head_1018 =
deleteNode_1018(head_1018, position_1018);
    //print list after
deletion
    System.out.println("List
    setelah posisi 2 dihapus: ");
    printList_1018(head_1018);
}
}

```

Gambar 2.4.1 Kode program HapusSLL_2511531018.

2.4.2 Langkah Kerja

2.4.2.1 Jika list kosong, maka langsung selesai (tidak ada yang dihapus). Jika ada isi, head digeser ke node berikutnya dan node pertama otomatis “kehapus” (karena ga direferensiin lagi), lalu balikin ke head baru

```

public static NodeSLL_2511531018
deleteHead_1018(NodeSLL_2511531018 head_1018) {
    //jika SLL kosong
    if (head_1018 == null)
        return null;
    //pindahkan head ke node berikutnya
    head_1018 = head_1018.next_1018;
    //return head baru
    return head_1018; }

```

Gambar 2.4.2.1 Fungsi deleteHead_1018 (hapus node pertama).

2.4.2.2 Jika list kosong dan cuma 1 node, maka langsung kosong (node itu dihapus).

```

public static NodeSLL_2511531018
removeLastNode_1018(NodeSLL_2511531018 head_1018) {
    //jika list kosong, return null
    if (head_1018 == null) {
        return null;
    }
}

```

Gambar 2.4.2.2 Fungsi removeLastNode_1018 (hapus node terakhir).

2.4.2.3 Mencari node sebelum terakhir, berhenti di node ke-2 terakhir. Pointer diputus dan node terakhir hilang.

```
NodeSLL_2511531018 secondLast_1018 = head_1018;
    while
(secondLast_1018.next_1018 != null) {
        secondLast_1018 =
secondLast_1018.next_1018;
    }
    //hapus node terakhir
secondLast_1018.next_1018 = null;
return head_1018; }
```

Gambar 2.4.2.3 Fungsi removeLastNode_1018 (hapus node terakhir).

2.4.2.4 Inisialisasi.

```
public static NodeSLL_2511531018
deleteNode_1018(NodeSLL_2511531018 head_1018, int
position_1018) {
    NodeSLL_2511531018 temp_1018 =
head_1018;
    NodeSLL_2511531018 prev_1018 =
null;
```

Gambar 2.4.2.4 Fungsi deleteNode_1018 (hapus di posisi tertentu).

2.4.2.5 List kosong dan hapus posisi 1 (head).

```
//jika linked list null
    if (temp_1018 == null)
        return head_1018;
    //kasus 1: head dihapus
    if (position_1018 == 1) {
        head_1018 =
temp_1018.next_1018;
        return head_1018; }
```

Gambar 2.4.2.5 Fungsi deleteNode_1018 (hapus di posisi tertentu).

2.4.2.6 Jika hapus ditengah, loop sampai node yang mau dihapus.

```
for (int i_1018 = 1; temp_1018 != null && i_1018 <
position_1018; i_1018++) {
    prev_1018 =
temp_1018;
```

```
temp_1018 =
temp_1018.next_1018; }
```

Gambar 2.4.2.6 Fungsi deleteNode_1018 (hapus di posisi tertentu).

- 2.4.2.7 Node sebelumnya langsung lompat ke node setelahnya dan node ditengah kehapus dan jika posisi melebihi panjang list, maka "Data tidak ada".

```
if (temp_1018 != null) {
    prev_1018.next_1018
= temp_1018.next_1018;
} else {
    System.out.println("Data tidak ada"); }
return head_1018; }
```

Gambar 2.4.2.7 Fungsi deleteNode_1018 (hapus di posisi tertentu).

- 2.4.2.8 Menampilkan isi linked list.

```
while (curr_1018.next_1018 != null) {
    System.out.print(curr_1018.data_1018+"-->");
    curr_1018 =
curr_1018.next_1018; }
if
(curr_1018.next_1018 == null) {
    System.out.print(curr_1018.data_1018); }
```

Gambar 2.4.2.8 Fungsi printList_1018.

- 2.4.2.9 Membuat linked list dan menampilkan list awal.

```
NodeSLL_2511531018 head_1018 = new
NodeSLL_2511531018(1);
head_1018.next_1018
= new NodeSLL_2511531018(2);
head_1018.next_1018.next_1018 = new
NodeSLL_2511531018(3);
head_1018.next_1018.next_1018.next_1018 = new
NodeSLL_2511531018(4);
head_1018.next_1018.next_1018.next_1018.next_1018 =
new NodeSLL_2511531018(5);
head_1018.next_1018.next_1018.next_1018.next_1018.ne
xt_1018 = new NodeSLL_2511531018(6);
// cetak list awal
System.out.println("list awal: ");
```

```
printList_1018(head_1018);
```

Gambar 2.4.2.9 Fungsi main.

2.4.2.10 Node 1 dihapus, head pindah ke 2.

```
head_1018 = deleteHead_1018(head_1018);

System.out.println("List setelah simpul terakhir di
hapus: ");

printList_1018(head_1018);
```

Gambar 2.4.2.10 Fungsi main.

2.4.2.11 Menghapus node terakhir, node 6 dihapus.

```
//hapus node terakhir
head_1018 =
removeLastNode_1018(head_1018);

System.out.println("List setelahsimpul terakhir di
hapus: ");

printList_1018(head_1018);
```

Gambar 2.4.2.11 Fungsi main.

2.4.2.12 Menghapus node di posisi ke-2.

```
int position_1018 = 2;
head_1018 =
deleteNode_1018(head_1018, position_1018);
//print list after
deletion

System.out.println("List setelah posisi 2 dihapus:
");

printList_1018(head_1018);
}
```

Gambar 2.4.2.12 Fungsi main.

2.4.3 Output

```
list awal:
1-->2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5-->6
List setelahsimpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 2.4.3 Output HapusSLL_2511531018.

2.6 Tugas

2.6.1 Class Pasien_2511531018

```
package pekan5_2511531018;

public class Pasien_2511531018 {
    String nama_1018;
    String keluhan_1018;
    int nomor_1018;
    Pasien_2511531018 next_1018;

    public Pasien_2511531018(String nama_1018, String
keluhan_1018, int nomor_1018) {
        this.nama_1018 = nama_1018;
        this.keluhan_1018 = keluhan_1018;
        this.nomor_1018 = nomor_1018;
        this.next_1018 = null;
    }
}
```

Gambar 2.6.1 Kode Program Pasien_2511531018.

2.6.2 Class RumahSakit_2511531018

```
package pekan5_2511531018;

import java.util.Scanner;

public class RumahSakit_2511531018 {
    // variabel untuk menyimpan nomor antrian
    otomatis
    static int counter_1018 = 0;

    // fungsi menambahkan pasien ke akhir linked list
    public static Pasien_2511531018
insertPasien_1018(Pasien_2511531018 head_1018,
String nama_1018, String keluhan_1018) {
        // nomor antrian bertambah setiap pasien
baru masuk
        counter_1018++;
        // membuat node pasien baru
        Pasien_2511531018 newNode_1018 = new
Pasien_2511531018(nama_1018, keluhan_1018,
counter_1018);

        // jika linked list kosong maka node baru
menjadi head
        if (head_1018 == null) {
```

```

        return newNode_1018;
    }
    // variabel sementara untuk menelusuri
linked list
    Pasien_2511531018 temp_1018 = head_1018;
    while (temp_1018.next_1018 != null) {
        temp_1018 = temp_1018.next_1018;
    }
    // menghubungkan node terakhir dengan
node baru
    temp_1018.next_1018 = newNode_1018;
    // mengembalikan head linked list
    return head_1018;
}
// fungsi memanggil pasien pertama
public static Pasien_2511531018
panggilPasien_1018(Pasien_2511531018 head_1018) {
    // jika linked list kosong
    if (head_1018 == null) {
        System.out.println("Antrian
kosong!");
        return null;
    }
    // menampilkan data pasien yang dipanggil
    System.out.println("Memanggil:");
    System.out.println("Nama      : " +
head_1018.nama_1018);
    System.out.println("Keluhan   : " +
head_1018.keluhan_1018);
    System.out.println("No Antrian: " +
head_1018.nomor_1018);
    // menggeser head ke node berikutnya
    return head_1018.next_1018;
}
// fungsi menampilkan seluruh data pasien
public static void
tampilkan_1018(Pasien_2511531018 head_1018) {
    // jika linked list kosong
    if (head_1018 == null) {
        System.out.println("Antrian
kosong!");
        return;
    }
    // variabel untuk traversal linked list
    Pasien_2511531018 curr_1018 = head_1018;
    // menelusuri node dari head sampai null
    while (curr_1018 != null) {

```



```

System.out.println(curr_1018.nomor_1018 + ". " +
curr_1018.nama_1018 + " - " +
curr_1018.keluhan_1018);
    curr_1018 = curr_1018.next_1018;
}
}
// fungsi mencari pasien berdasarkan nama
public static void
cari_1018(Pasien_2511531018 head_1018, String
namaCari_1018) {
    // variabel untuk traversal linked list
    Pasien_2511531018 curr_1018 = head_1018;
    // menelusuri linked list sampai data
ditemukan
    while (curr_1018 != null) {
        if
(curr_1018.nama_1018.equalsIgnoreCase(namaCari_1018
)) {
            System.out.println("Pasien
ditemukan:");
            System.out.println("Nama: " +
curr_1018.nama_1018);
            System.out.println("Keluhan: " +
curr_1018.keluhan_1018);
            return;
        }
        // pindah ke node berikutnya
        curr_1018 = curr_1018.next_1018;
    }
    // jika data tidak ditemukan
    System.out.println("Pasien tidak
ditemukan!");
}
// fungsi mengecek status antrian
public static void
status_1018(Pasien_2511531018 head_1018) {
    // jika linked list kosong
    if (head_1018 == null) {
        System.out.println("Antrian
kosong!");
        return;
    }

    int jumlah_1018 = 0;
    // variabel traversal
    Pasien_2511531018 curr_1018 = head_1018;

```

```

// menghitung jumlah node pada linked
list
while (curr_1018 != null) {
    jumlah_1018++;
    curr_1018 = curr_1018.next_1018;
}
// menampilkan jumlah pasien dan pasien
terdepan
System.out.println("Jumlah pasien : " +
jumlah_1018);
System.out.println("Pasien terdepan : " +
head_1018.nama_1018);
}
// program utama
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    // head awal linked list bernilai null
    Pasien_2511531018 head_1018 = null;
    int pilih_1018;

    do {
        System.out.println("\n=== ANTRIAN
RUMAH SAKIT ===");
        System.out.println("1. Daftarkan
Pasien : ");
        System.out.println("2. Panggil Pasien
: ");
        System.out.println("3. Tampilkan
Antrian : ");
        System.out.println("4. Cari Pasien
: ");
        System.out.println("5. Cek Status
Antrian : ");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilih_1018 = input.nextInt();
        input.nextLine();

        switch (pilih_1018) {
            case 1:
                System.out.print("Masukka
Nama Pasien : ");
                String nama_1018 =
input.nextLine();
                System.out.print("Masukkan
Keluhan : ");

```

```

                                String keluhan_1018 =
input.nextLine();
                                // menambahkan pasien ke
linked list
                                head_1018 =
insertPasien_1018(head_1018, nama_1018,
keluhan_1018);

                                System.out.println("Pasien
berhasil didaftarkan! Nomor Antrian: " +
counter_1018);
                                break;

                                case 2:
                                // menghapus node pertama
                                head_1018 =
panggilPasien_1018(head_1018);
                                break;

                                case 3:
                                // menampilkan seluruh linked
list
                                tampilkan_1018(head_1018);
                                break;

                                case 4:
                                System.out.print("Masukkan
Nama yang dicari : ");
                                String cari_1018 =
input.nextLine();
                                // mencari data pasien
                                cari_1018(head_1018,
cari_1018);
                                break;

                                case 5:
                                // mengecek status linked list
                                status_1018(head_1018);
                                break;

                                case 6:

                                System.out.println("Terimakasih!");
                                break;

                                default:

```

```

                                System.out.println("Pilihan
tidak valid!");
                                }

                                } while (pilih_1018 != 6);
                                }

                                }

```

Gambar 2.6.2 Kode program RumahSakit_2511531018.

2.6.3 Output

1. Menambahkan pasien.

```

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien      :
2. Panggil Pasien        :
3. Tampilkan Antrian     :
4. Cari Pasien           :
5. Cek Status Antrian    :
6. Keluar
Pilihan: 1
Masukka Nama Pasien      : syasya
Masukkan Keluhan         : sakit kepala
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien      :
2. Panggil Pasien        :
3. Tampilkan Antrian     :
4. Cari Pasien           :
5. Cek Status Antrian    :
6. Keluar
Pilihan: 1
Masukka Nama Pasien      : james
Masukkan Keluhan         : demam
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien      :
2. Panggil Pasien        :
3. Tampilkan Antrian     :
4. Cari Pasien           :
5. Cek Status Antrian    :
6. Keluar
Pilihan: 1
Masukka Nama Pasien      : martin
Masukkan Keluhan         : flu ringan
Pasien berhasil didaftarkan! Nomor Antrian: 3

```

Output diatas menunjukkan bagaimana program dapat menafrakan nama pasien dengan memilih menu 1. User diminta untuk mengisi nama pasien serta keluhan yang dialami oleh pasien. Setelah data di input, maka pasien berhasil didaftarkan dan mendapat nomor antrian sesuai dengan urutan daftar pasien.

2. Menampilkan antrian.

```

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien      :
2. Panggil Pasien        :
3. Tampilkan Antrian     :
4. Cari Pasien           :
5. Cek Status Antrian    :
6. Keluar
Pilihan: 3
1. syasya - sakit kepala
2. james - demam
3. martin - flu ringan
4. keonho - terkilir

```

Output diatas menunjukkan bagaimana program dapat menampilkan antrian yang tersedia. Antrian tersebut berisi pasien-pasien yang telah didaftarkan sebelumnya.

3. Hbedygdeuygqde

```

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien      :
2. Panggil Pasien        :
3. Tampilkan Antrian     :
4. Cari Pasien           :
5. Cek Status Antrian    :
6. Keluar
Pilihan: 4
Masukkan Nama yang dicari : martin
Pasien ditemukan:
Nama: martin
Keluhan: flu ringan

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien      :
2. Panggil Pasien        :
3. Tampilkan Antrian     :
4. Cari Pasien           :
5. Cek Status Antrian    :
6. Keluar
Pilihan: 4
Masukkan Nama yang dicari : awa
Pasien tidak ditemukan!

```

Output diatas menunjukkan bagaimana program dapat mencari pasien berdasarkan nama. User dapat mencari pasien dengan menginput nama pasien, jika pasien ada dalam antrian, maka pasien ditemukan dan program mencetak nama pasien serta keluhan yang dialami oleh pasien. Namun, jika nama pasien tidak ditemukan, maka pasien telah dipanggil atau pasien tidak terdaftar dalam antrian.

4. Panggil pasien.

```

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien :
2. Panggil Pasien :
3. Tampilkan Antrian :
4. Cari Pasien :
5. Cek Status Antrian :
6. Keluar
Pilihan: 2
Memanggil:
Nama : syasya
Keluhan : sakit kepala
No Antrian: 1

```

Output diatas menunjukkan program dapat memanggil pasien mulai dari antrian yang paling atas. Sistem ini menggunakan First In, First Out (FIFO). Ketika pasien dipanggil, program menampilkan nama pasien, keluhan, serta nomor antrian pasien.

5. Cek status antrian.

```

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien :
2. Panggil Pasien :
3. Tampilkan Antrian :
4. Cari Pasien :
5. Cek Status Antrian :
6. Keluar
Pilihan: 5
Jumlah pasien : 3
Pasien terdepan : james

```

Output diatas menunjukkan program dapat mengecek status antrian. Ketika user memilih menu no 5, maka program menampilkan jumlah pasien yang masih tersedia dalam antrian. Program juga menampilkan pasien terdepan, yang dimana pasien berada pada antrian paling awal. Sehingga ketika user memanggil pasien, maka pasien tersebut yang dipanggil di awal.

6. Panggil pasien.

```

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien :
2. Panggil Pasien :
3. Tampilkan Antrian :
4. Cari Pasien :
5. Cek Status Antrian :
6. Keluar
Pilihan: 2
Antrian kosong!

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien :
2. Panggil Pasien :
3. Tampilkan Antrian :
4. Cari Pasien :
5. Cek Status Antrian :
6. Keluar
Pilihan: 3
Antrian kosong!

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien :
2. Panggil Pasien :
3. Tampilkan Antrian :
4. Cari Pasien :
5. Cek Status Antrian :
6. Keluar
Pilihan: 5
Antrian kosong!

```

Output diatas menunjukkan bahwa program dapat menampilkan antrian kosong apabila tidak ada pasien dalam antrian.

7. Keluar.

```
=== ANTRIAN RUMAH SAKIT ===  
1. Daftarkan Pasien      :  
2. Panggil Pasien        :  
3. Tampilkan Antrian     :  
4. Cari Pasien           :  
5. Cek Status Antrian    :  
6. Keluar                :  
Pilihan: 6  
Terimakasih!
```

Untuk menu no 6 berfungsi sebagai berhentinya program yang dijalankan.

BAB III PENUTUP

Single Linked List (SLL) merupakan struktur data dinamis yang tersusun dari node-node yang saling terhubung melalui pointer. Setiap node memiliki data dan referensi ke node berikutnya, sehingga memungkinkan pengelolaan data secara fleksibel. Melalui penggunaan class NodeSLL, dapat dibangun struktur dasar linked list. Operasi penambahan node yang diimplementasikan dalam TambahSLL menunjukkan bahwa data dapat disisipkan di berbagai posisi tanpa perlu menggeser elemen lain. Selanjutnya, proses pencarian pada PencarianSLL dilakukan dengan menelusuri node secara berurutan hingga data ditemukan. Sementara itu, pada HapusSLL, penghapusan node dilakukan dengan mengubah hubungan antar node sehingga struktur list tetap terjaga. Secara keseluruhan, pemahaman terhadap operasi penambahan, pencarian, dan penghapusan pada Single Linked List membantu dalam mengelola data secara lebih efisien dan menjadi dasar untuk mempelajari struktur data yang lebih kompleks.

DAFTAR PUSTAKA

- [1] A. Kadir, *Algoritma dan Struktur Data dalam Bahasa Java*. Yogyakarta: Penerbit Andi, 2018.
- [2] Rosa A.S dan M. Shalahuddin, *Struktur Data dan Pemrograman Java*. Bandung: Informatika Bandung, 2019.
- [3] Sutarman, *Struktur Data*. Yogyakarta: Graha Ilmu, 2012.
- [4] Wahana Komputer, *Pemrograman Java Tingkat Lanjut*. Semarang: Penerbit Andi, 2015.
- [5] R. Munir, *Konsep dan Implementasi Struktur Data*. Bandung: Informatika Bandung, 2016.